

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчёт по лабораторной работе №4
«Комбинаторный код»

Выполнила работу:
Абаянцева Евгения
Академическая группа №J3110
Принято:
...

Санкт-Петербург
2024 г.

Введение

Цель – найти все возможные комбинации из заданных элементов в массиве.

Задача: Дан массив размера N целых чисел. В нем необходимо найти все подмассивы размера k , сумма элементов которых равна 0. В массиве доступны целые положительные и отрицательные числа, а так же 0. N – до 25, k – 5.

Теоретическая подготовка

Данная задача предполагает решение с использованием комбинаторного кода. Код предполагает несколько циклов `for` внутри друг друга. Используемые типы данных для этой задачи – целые числа (тип `int`), массив (тип `vector`).

Реализация

Для этой задачи требуются следующие библиотеки:

1. Библиотека `iostream` – эта основная библиотека C++ для ввода и вывода текста с консоли;
2. Библиотека `vector` – эта библиотека определяет вектор шаблона класса контейнера и несколько вспомогательных шаблонов;
3. Библиотека `chrono` – эта библиотека отвечает за работу со временем и датой.

Реализация была выполнена через комбинаторный код, используя пять циклов `for`. Такой подход был выбран для того, чтобы проверить каждый подмассив размера k (в случае задачи, 5) с элементами из исходного массива. В пятом цикле считается сумма такого подмассива и сравнивается с нулём. Если текущая сумма равняется нулю, то такой подмассив записывается в массив, который в конце концов является ответом задачи.

Особенность реализации: компактность кода и более удобное вычисление суммы различных элементов массива.

Экспериментальная часть

1. Подсчёт памяти: 1 массив (16 байт), 2 массива и 2 int (82 байт).Всего в сумме – 98 байт.

2. Асимптотика: $O(n^5)$.

3. График зависимости времени от числа элементов(см. рис. 1 внизу Приложения). Таблица результатов, по которым строился график:

Таблица 1 – Результаты тестирования алгоритма

Размер массива	Время, сек
10	0,0012599
11	0,0023789
12	0,001956
13	0,0031623
14	0,0137606
15	0,0207931
16	0,0652373

Заключение

В ходе выполнения этой задачи был реализован комбинаторный код. Цель работы была достигнута путём проверки кода через созданные тесты.

В качестве дальнейшего исследования можно предложить оптимизацию алгоритма с точки зрения уменьшения циклов (вместо пяти – четыре).

Приложения

```
1 #include <iostream>
2 #include <chrono>
3 #include <vector>
4
5 std::vector<std::vector<int>> findSublists( const std::vector<int>& nums, int
   target, int& coun ) {
6     std::vector<std::vector<int>> array;
7     for (size_t i_1 = 0; i_1 < nums.size() - 4; i_1++) {
8         int sum_so_far = 0;
9         for (size_t i_2 = i_1 + 1; i_2 < nums.size() - 3; i_2++) {
10             for (size_t i_3 = i_2 + 1; i_3 < nums.size() - 2; i_3++) {
11                 for (size_t i_4 = i_3 + 1; i_4 < nums.size() - 1; i_4++) {
12                     for (size_t i_5 = i_4 + 1; i_5 < nums.size(); i_5++) {
13                         sum_so_far = nums[i_1] + nums[i_2] + nums[i_3] + nums[i_4] + nums[
14 i_5];
15                         if (sum_so_far == target) {
```

```

15         array.push_back({ static_cast<int>(i_1), static_cast<int>(i_2),
16             static_cast<int>(i_3), static_cast<int>(i_4), static_cast<int>(i_5) });
17     }
18 }
19 }
20 }
21 }
22
23 coun += sizeof(array);
24
25 std::cout << coun << "bt\n";
26 return array;
27 }
28
29 bool checkout(std::vector<std::vector<int>> mn_array, std::vector<int> nums, int
    & coun) {
30     bool flag = true;
31     coun += sizeof(flag);
32
33     for (const auto& indices : mn_array) {
34         // std::cout << "[";
35         int sum_on = 0;
36         for (size_t i = 0; i < indices.size(); ++i) {
37             sum_on += nums[indices[i]];
38             // std::cout << indices[i];
39             if (i < indices.size() - 1) {
40                 // std::cout << ", ";
41             }
42         }
43         // std::cout << "]" << std::endl;
44
45         if (sum_on != 0)
46             flag = false;
47     }
48
49     return flag;
50 }
51
52 void test1(int coun) {
53     int target = 0;
54     auto start = std::chrono::steady_clock::now();
55
56     std::vector<int> nums = { -2, 3, 4, -7, 1, 2, 3, 3, 1, 4, 0, 1, 2, 7, -3, 0 };
57     auto mn_array = findSublists(nums, target, coun);
58     //struct tm *now = localtime(&mytime);
59
60     auto end = std::chrono::steady_clock::now();
61     std::chrono::duration<double> elapsed = end - start;
62     std::cout << "Time: " << elapsed.count() << "\n";
63
64     if (checkout(mn_array, nums, coun))
65         std::cout << "Test passed\n";
66     else
67         std::cout << "Test not passed\n";
68
69     coun += sizeof(nums) + sizeof(mn_array) + sizeof(target) + sizeof(coun);
70     std::cout << coun << "bt\n";
71 }
72

```

```

73 void test2(int coun) {
74     int target = 0;
75     std::vector<int> nums = { 1, -1, 2, -2, 0, 3, -3, 2, -3, 2, 3, -5 };
76     auto mn_array = findSublists(nums, target, coun);
77
78     if (checkout(mn_array, nums, coun))
79         std::cout << "Test passed\n";
80     else
81         std::cout << "Test not passed\n";
82
83     coun += sizeof(nums) + sizeof(mn_array) + sizeof(target) + sizeof(coun);
84     std::cout << coun << "bt\n";
85 }
86
87 int main() {
88     int coun = 0;
89     // std::vector<int> nums = { 3, 4, -7, 1, 3, 3, 1, 4, 3, 1 };
90     // int target = 0;
91
92     //std::cout << "Input array: ";
93     //for (const auto& num : nums) {
94     //    std::cout << num << " ";
95     //}
96     test1(coun);
97     test2(coun);
98
99     // std::cout << coun << "bt";
100
101     return 0;
102 }

```

Листинг 1 – код файла lab 4.cpp

График зависимости времени от числа элементов:

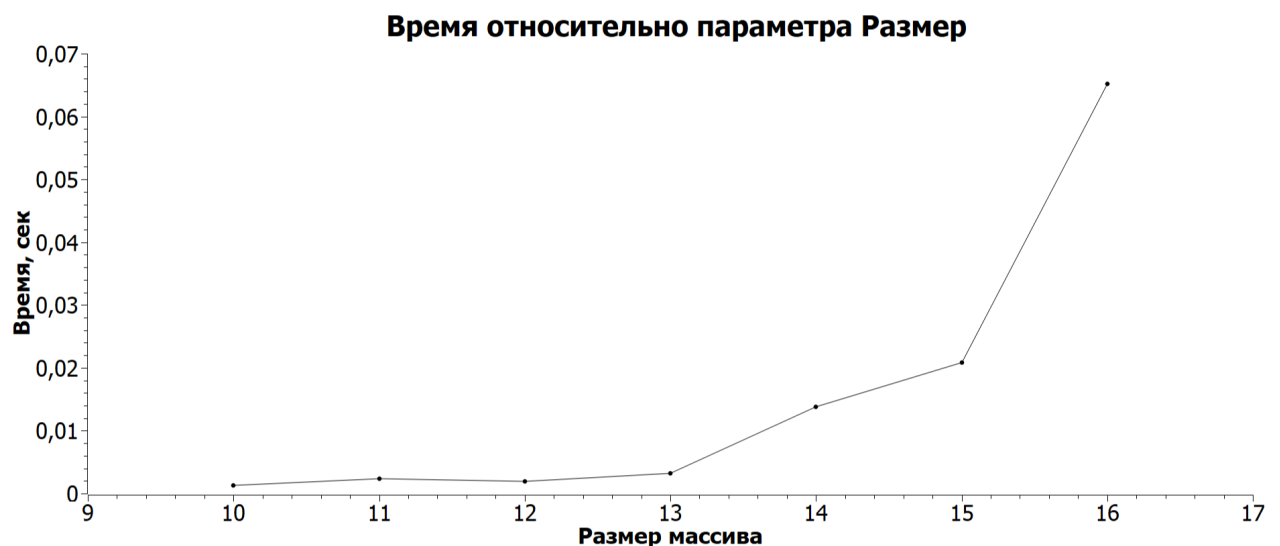


Рис. 1 – График зависимости времени от числа элементов